Modern Storages and Data Warehousing
Week 3 - MPP

Попов Илья, i.popov@hse.ru

Recap прошлых занятий

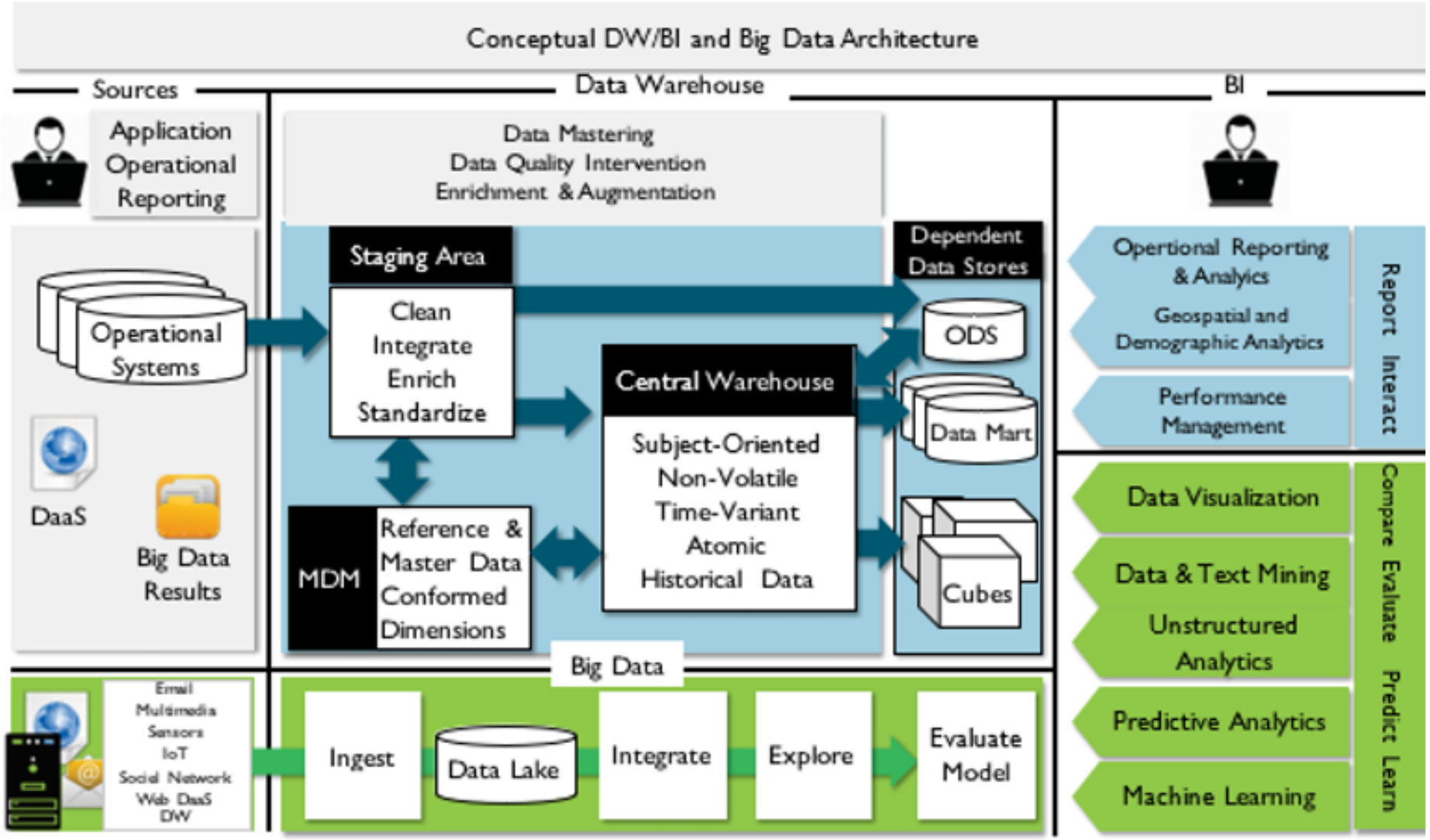


Figure 5: Date Warehouse Concept

1 - Mass Parallel Processing

Мотивация

〉 Мы учимся эффективно хранить и обрабатывать большие данные
〉 Наши данные перестали помещаться в один хост - дальше раздувать некуда
〉 Нам не хватает вычислительных мощностей одной машины для того, чтобы обрабатывать данные с требуемой скоростью
〉 А что если распараллелить вычисления, но сделать так, чтобы данные обрабатывали хосты, похожие на классические СУБД?

Мотивация

〉 Мы учимся эффективно хранить и обрабатывать большие данные
〉 Наши данные перестали помещаться в один хост - дальше раздувать некуда
〉 Нам не хватает вычислительных мощностей одной машины для того, чтобы обрабатывать данные с требуемой скоростью
〉 А что если распараллелить вычисления, но сделать так, чтобы данные обрабатывали хосты, похожие на классические СУБД?

〉 Так вот оказывается, что так пытались делать чуть ли не с середины 1980-х - такие решения называнию Mass Parallel Processing базами, или MPP

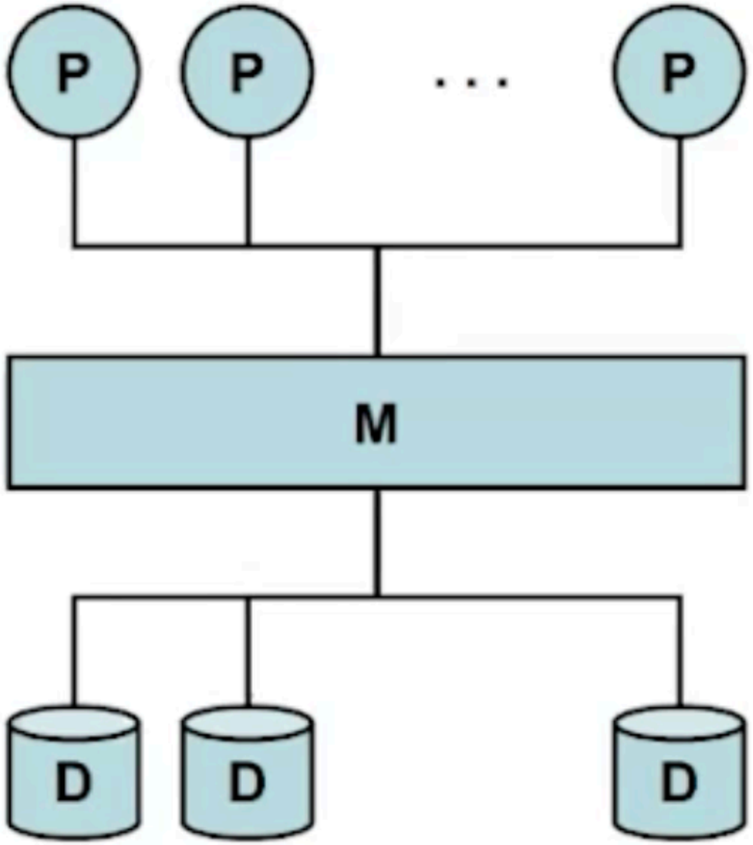
Лирическое отступление

Мы хотим создать распределенную СУБД для эффективного хранения и обработки данных.

Какие тут могут быть подходы:

- SE - Shared Everything - хосты делят все ресурсы (CPU, RAM, Disk)
- SD - Shared Disks - хосты делят только диски
- SN - Shared Nothing - нет совместного использования ресурсов
- CD - Clustered Disk
- CE - Clustered Everything

SE (Shared Everything)



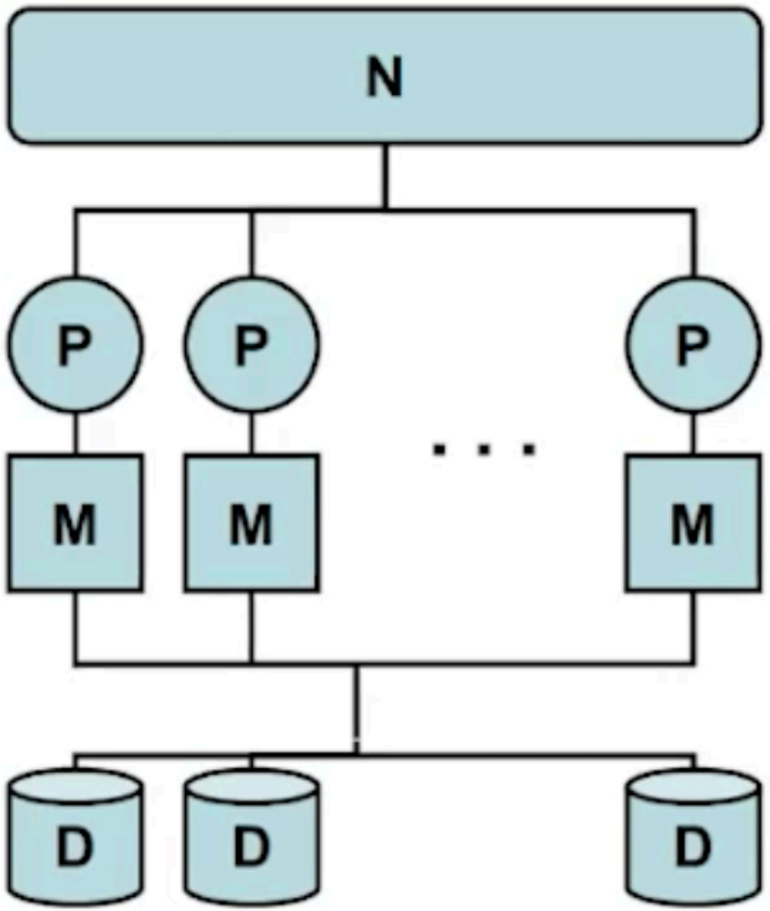
〉 У всех процессоров общая RAM

〉 Все диски доступны в равной степени и приоритете всем процессорам

〉 Межпроцессорные коммуникации - через RAM

〉 По сути - наш обычный компьютер

SD (Shared Disk)

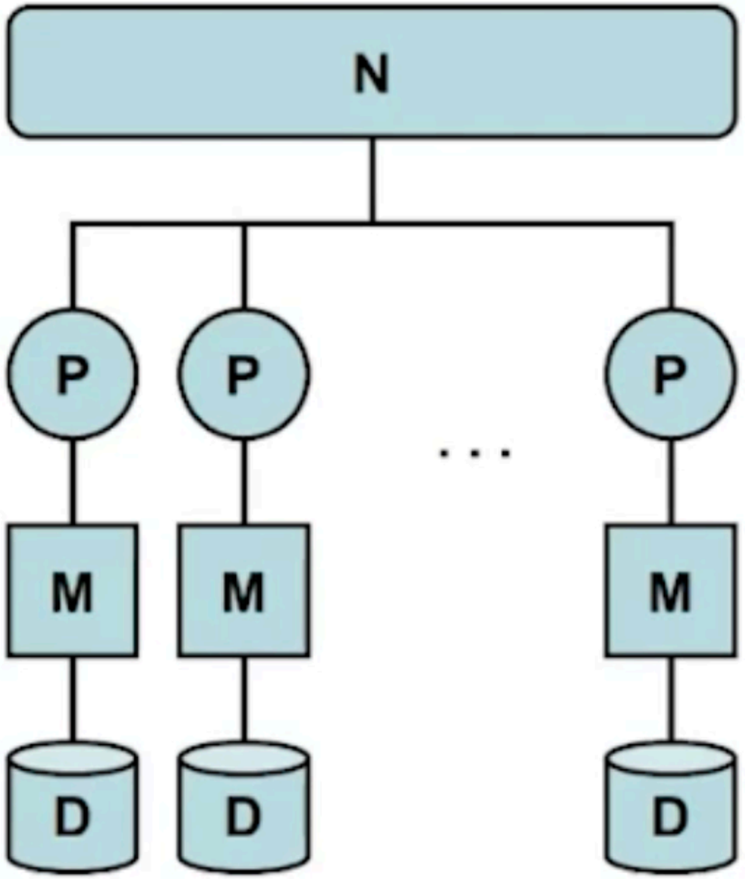


〉 У каждого процессора своя RAM

〉 Все диски доступны в равной степени и приоритете всем процессорам

〉 Межпроцессорные коммуникации - через сеть (interconnect)

SN (Shared Nothing)

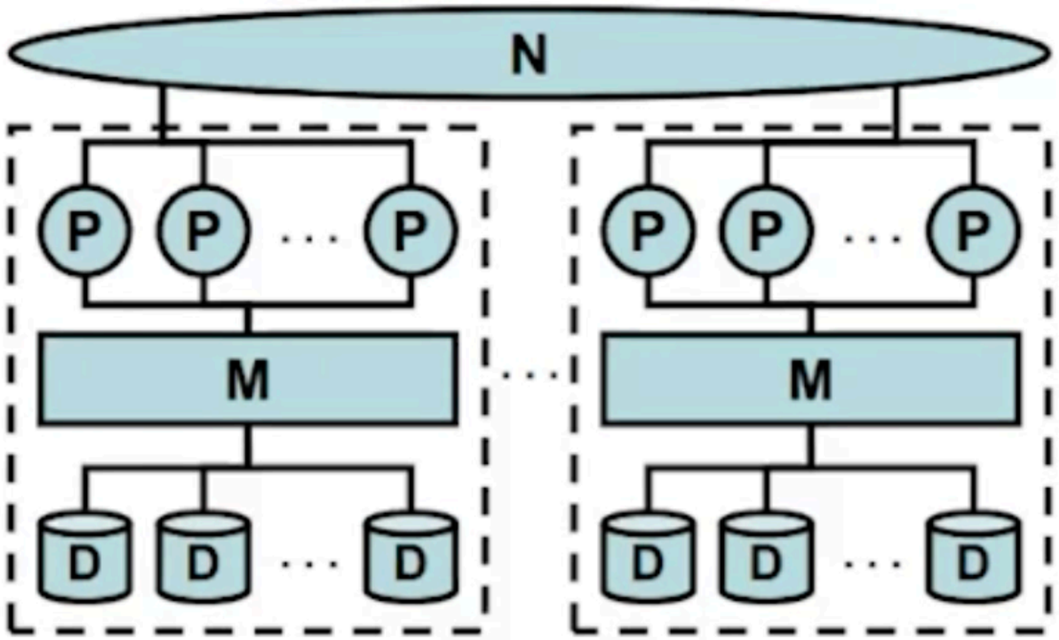


〉 Каждый процессор имеет свою оперативную память и диск

〉 Межпроцессорные коммуникации - через сеть (interconnect)

〉 Классическая архитектура MPP-СУБД (Teradata, Greenplum, Vertica Enterprise)

CE (Clustered Everything)

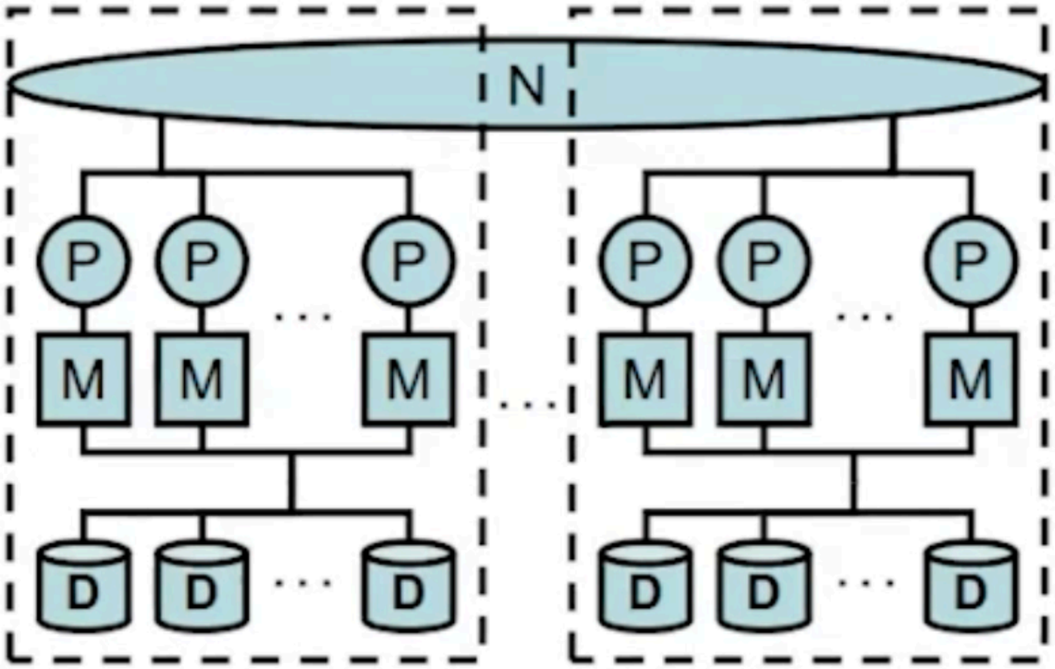


〉 Несколько SE-кластеров, связанных сетью

〉 Межпроцессорные коммуникации - через RAM

〉 Межкластерные коммуникации - через сеть (interconnect)

CD (Clustered Disk)



- Несколько SD-кластеров, связанных сетью
- Межпроцессорные коммуникации - через локальную сеть
- Межкластерные коммуникации - через глобальную сеть

2 - Архитектура и примеры

Фрагментарный параллелизм

Поговорим о концептах, которые схожи во всех указанных системах

Во всех перечисленных системах есть способ задать распределение по узлам:

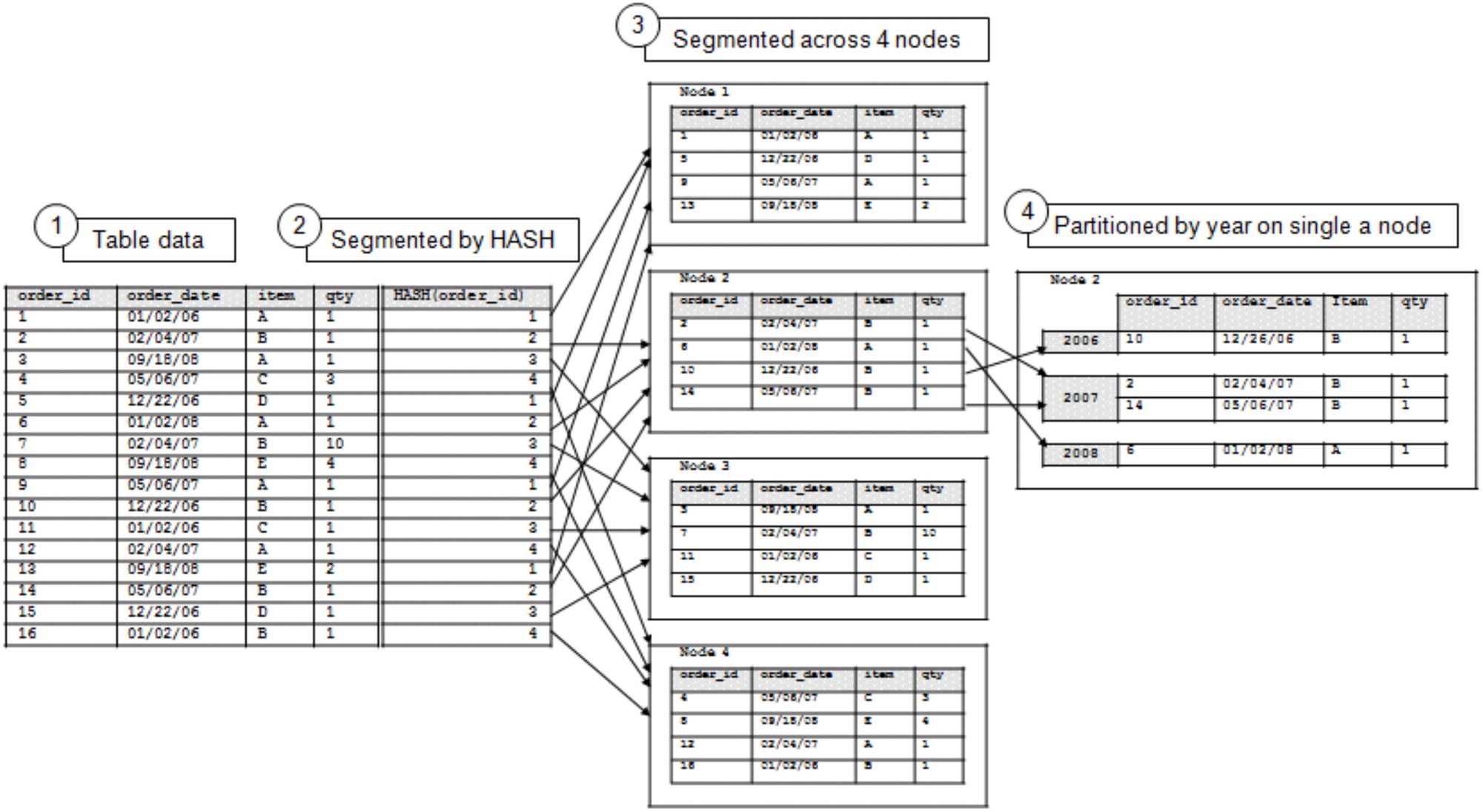
- Teradata - primary index (+ NoPI)
- Greenplum - distribution key (+ DISTRIBUTED RANDOMLY/REPLICATED)
- Vertica - segmentation key (+ UNSEGMENTED ALL NODES)

Партицирование

Физическое разделение данных на диске по какому-то полю

- Данные разделяются по нодам в зависимости от ключа сегментации
- В каждой ноде данные делятся на фрагменты по ключу партицирования
- Внутри партиции строки сортируются в зависимости от выбранного ключа сортировки
- Ключ партицирования может не совпадать с ключом сегментации

Сегментирование и партицирование вместе



order_id	order_date	item	qty	HASH(order_id)
1	01/02/06	A	1	1
2	02/04/07	B	1	2
3	09/18/08	A	1	3
4	05/06/07	C	3	4
5	12/22/06	D	1	1
6	01/02/08	A	1	2
7	02/04/07	B	10	3
8	09/18/08	E	4	4
9	05/06/07	A	1	1
10	12/22/06	B	1	2
11	01/02/06	C	1	3
12	02/04/07	A	1	4
13	09/18/08	E	2	1
14	05/06/07	B	1	2
15	12/22/06	D	1	3
16	01/02/06	B	1	4

Node 1

order_id	order_date	item	qty
1	01/02/06	A	1
5	12/22/06	D	1
9	05/06/07	A	1
13	09/18/08	E	2

Node 2

order_id	order_date	item	qty
2	02/04/07	B	1
6	01/02/08	A	1
10	12/22/06	B	1
14	05/06/07	B	1

Node 3

order_id	order_date	item	qty
3	09/18/08	A	1
7	02/04/07	B	10
11	01/02/06	C	1
15	12/22/06	D	1

Node 4

order_id	order_date	item	qty
4	05/06/07	C	3
8	09/18/08	E	4
12	02/04/07	A	1
16	01/02/06	B	1

Node 2

	order_id	order_date	Item	qty
2006	10	12/26/06	B	1
2007	2	02/04/07	B	1
	14	05/06/07	B	1
2008	6	01/02/08	A	1

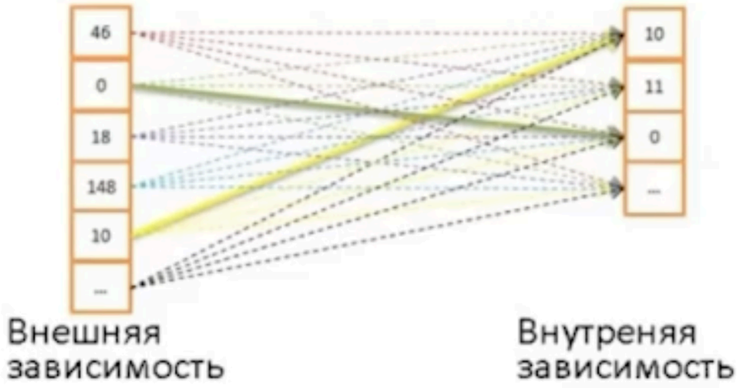
Виды JOIN

Был курс по базам?
Помним, какие виды JOIN бывают?

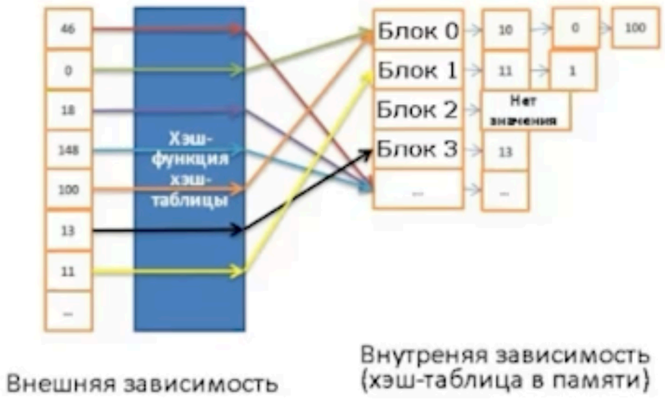


пингвин для привлечения внимания

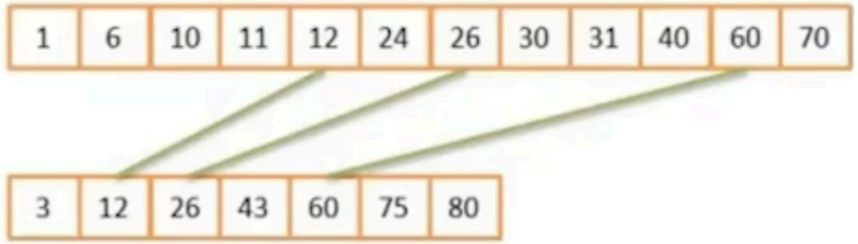
Виды JOIN



Nested loop

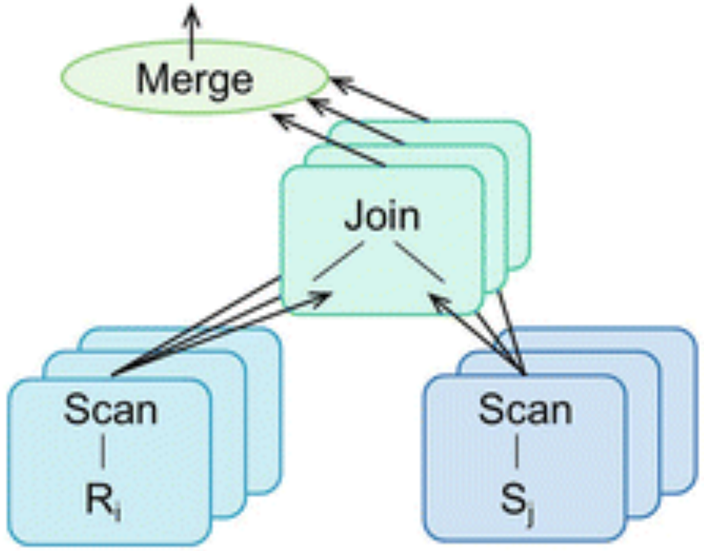


Hash join

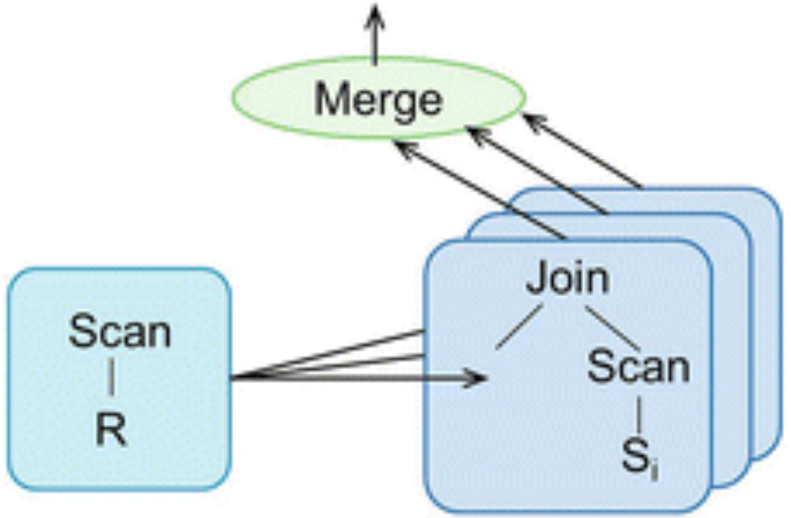


Merge join

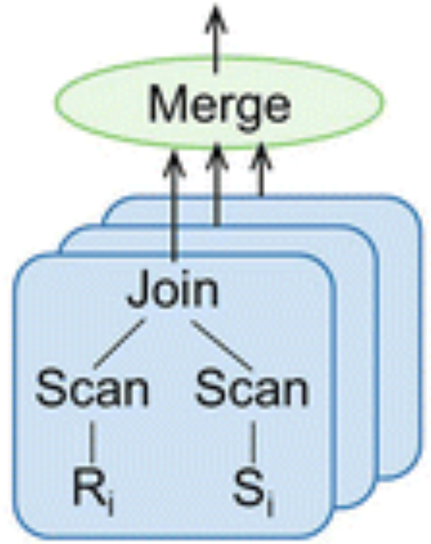
Виды JOIN



Redistribute



Broadcast



Co-locate

Local Merge join на MPP

〉Блоки таблицы читаются только один раз
〉Необходимо, чтобы данные были одинаково распределены
〉Необходимо, чтобы данные были одинаково отсортированы
〉Выгоден в Vertica при подходящих проекциях; в Greenplum по умолчанию hash join

Выбор стратегии join в MPP:
〉Если данные уже распределены по ключу join - co-located join
〉Иначе маленькую таблицу рассылаем на все узлы (broadcast), большую - перераспределяем по ключу (redistribute) + сортируем при необходимости
〉На каждом узле выполняем локальный hash, merge или nested loop join

Проблема выбора ключа сегментации

Критерии выбора ключа сегментации:

〉 Ключ часто участвует в JOIN / GROUP BY с другими таблицами

- Не дата/timestamp
- Не колонка частых WHERE-фильтров

〉 Распределение значений должно быть равномерным

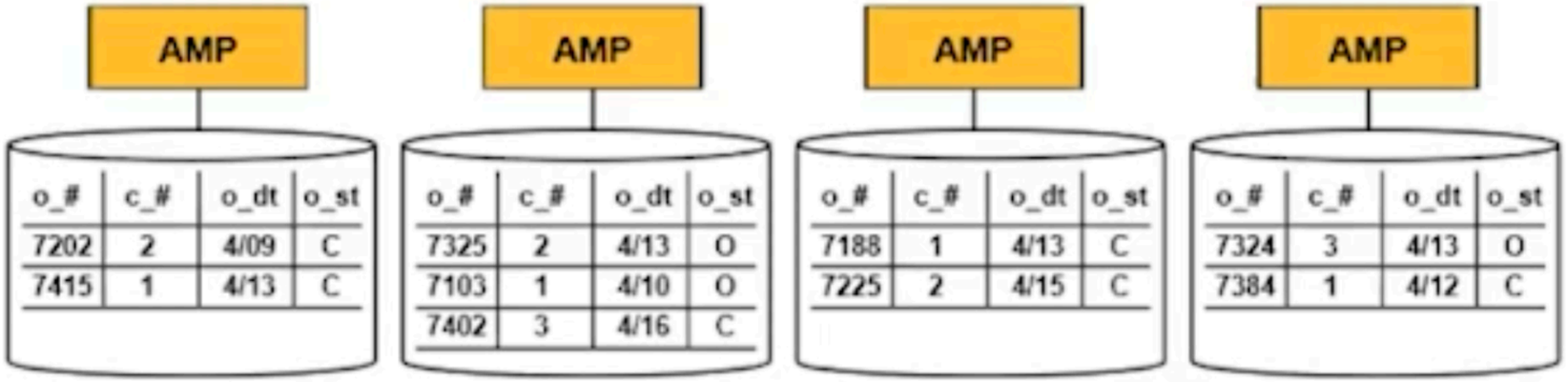
〉 В больших таблицах количество уникальных значений ключа должно превышать количество нод как минимум в 10 раз

〉 Изменения в данных ключа сегментации не должны происходить слишком часто

Пример

Orders

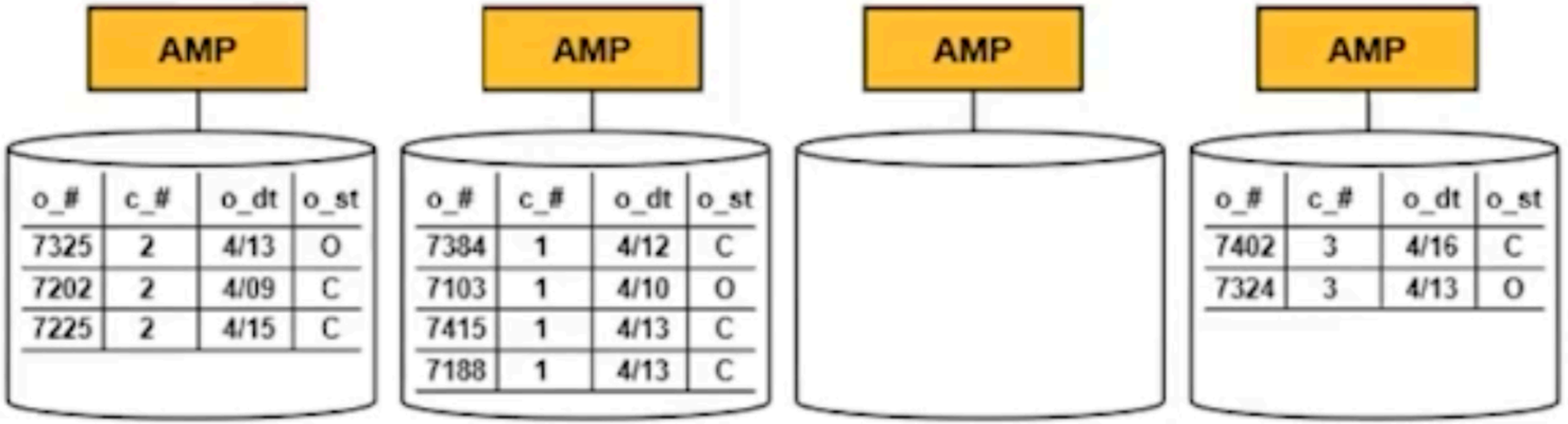
Order Number	Customer Number	Order Date	Order Status
PK			
UPI			
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Пример

Orders

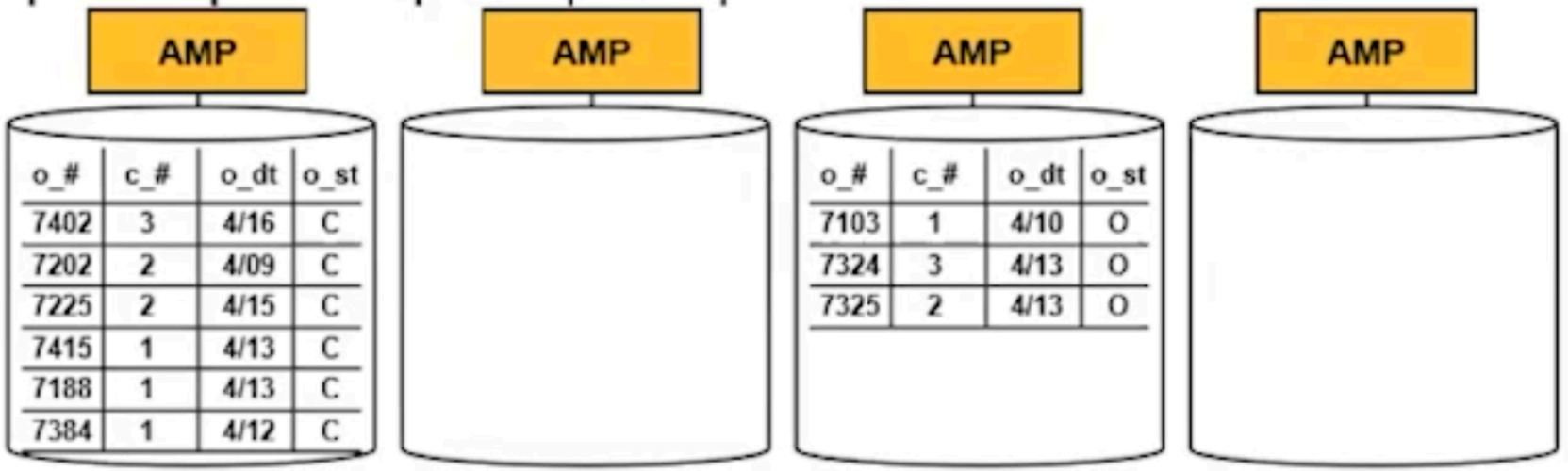
Order Number	Customer Number	Order Date	Order Status
PK			
	NUPI		
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Пример

Orders

Order Number	Customer Number	Order Date	Order Status
PK			
			NUPI
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Проблема использования партицирования

Плюсы:

〉 Если на большой таблице применено партицирование, то читается только необходимый блок данных

〉 Быстрое удаление и архивирование старых партиций

Минусы:

〉 Дополнительные накладные расходы на чтение/запись файлов

〉 Запросы, которые не используют колонки партицирования, работают хуже

〉 Каждая партиция - отдельные файлы, поэтому растут накладные расходы на открытие файлов и слияние потоков

Фрагментарный параллелизм

Во всех перечисленных системах есть способ задать распределение по узлам:

〉 Teradata - primary index (+ NoPI)
〉 Greenplum - distribution key (+ DISTRIBUTED RANDOMLY/REPLICATED)
〉 Vertica - segmentation key (+ UNSEGMENTED ALL NODES)

Методы доступа к данным:

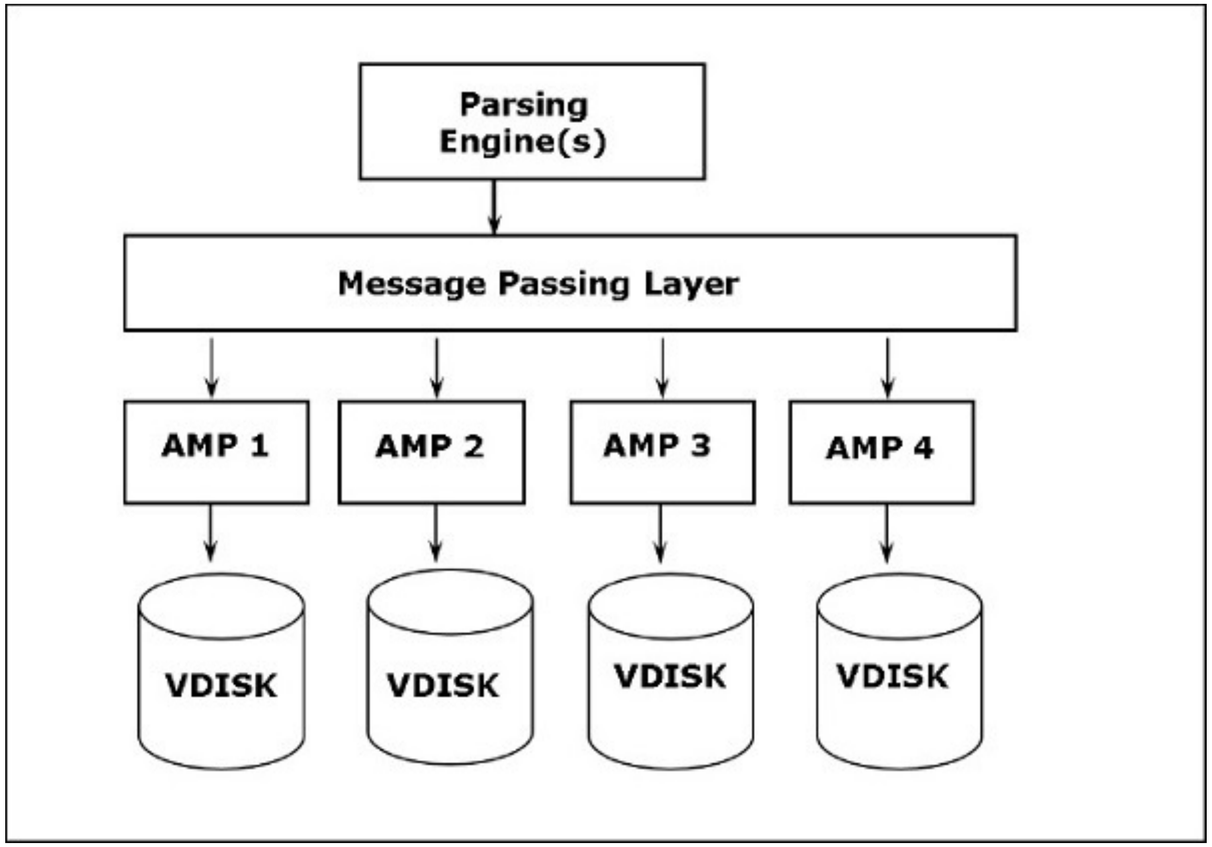
〉 Full scan - все перечисленные MPP
〉 Partition pruning - все перечисленные MPP
〉 Обращение по ключу сегментации - все перечисленные MPP
〉 Использование проекций - Vertica
〉 Использование вторичного индекса - Teradata, Greenplum

Teradata



- Первая из коммерческих систем такого класса
- Выкатили MPP еще в 1984 (DBC/1012) - с амбициозной целью обрабатывать терабайт данных и дать возможность горизонтального масштабирования
- В 1990-е буквально стандарт для больших OLAP’ов

Teradata



Parsing Engines:

〉 Управление сессиями пользователей

〉 Чтение и оптимизация запросов

〉 Выдача результатов пользователю

Access Module Processor:

〉 Управляет выделенным пространством

〉 Выполняет join, сортировки, агрегации и блокировки

Message Passing Layer:

〉 Посредник между PE и AMP

Virtual Disks:

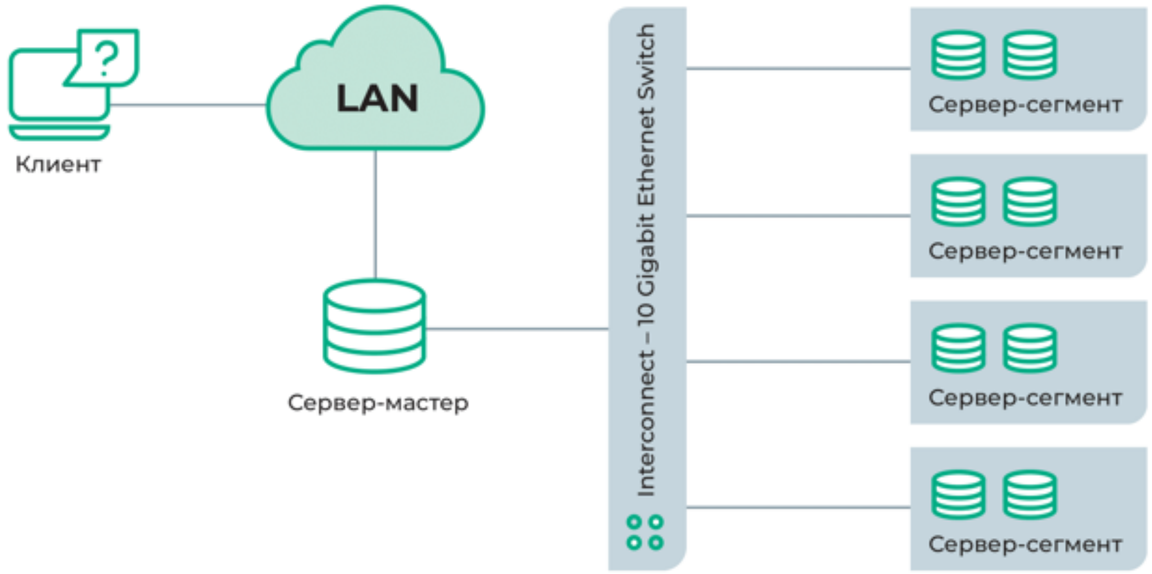
〉 Непосредственно выделенное дисковое пространство

Greenplum



- Начал разрабатываться в 2003 году
- Главная идея - взять зрелую RDBMS (PostgreSQL) и использовать её для горизонтального масштабирования
- Был проектом с открытым исходным кодом, из-за чего приобрел большую популярность в 2010-х
- В 2023 был поглащен Broadcom, после чего в мае 2024 был переведен на закрытую лицензию
- После этого в Apache Foundation вошла Cloudberry - продолжатель open source GP
- Есть и другие популярные форки: GreengageDB, WarehousePG

Greenplum



Pic. 3

- Здесь архитектурно все чуть хуже в плане отказоустойчивости: есть одна (с warm standby, failover ручной) мастер-нода - это точка отказа и бутылочное горлышко в системе
- Внутри сегментов по сути лежат PostgreSQL с репликами

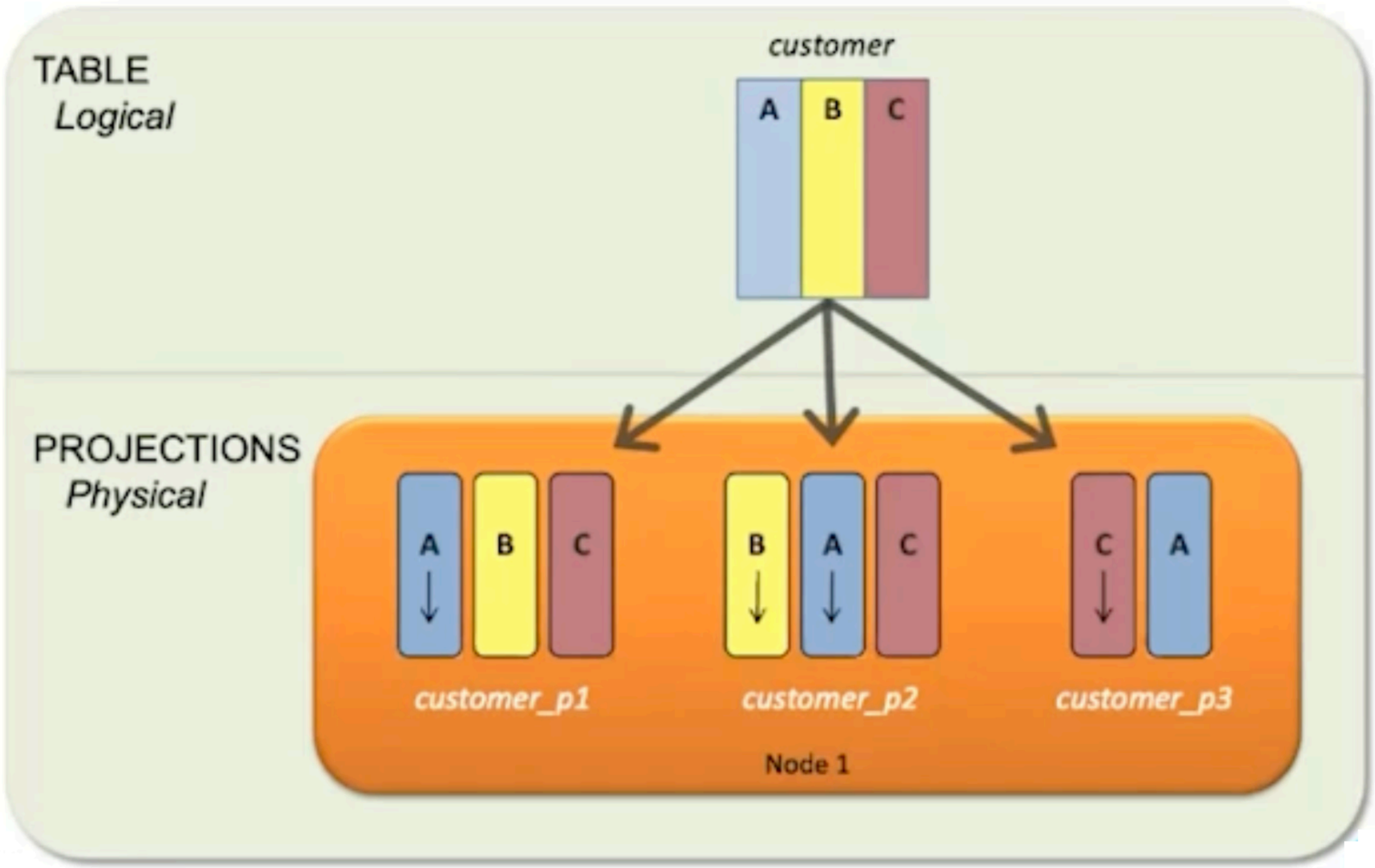
- Умеет как в локальные, так и в распределенные JOIN
- Полностью соответствует ACID
- Нативно поддерживает постгресовые приколы (расширения типа PostGIS, нативная репликация внутри сегментов)
- Умеет в масштабирование “на горячую” - можно добавить ноду в кластер и запустить ребалансировку

Vertica

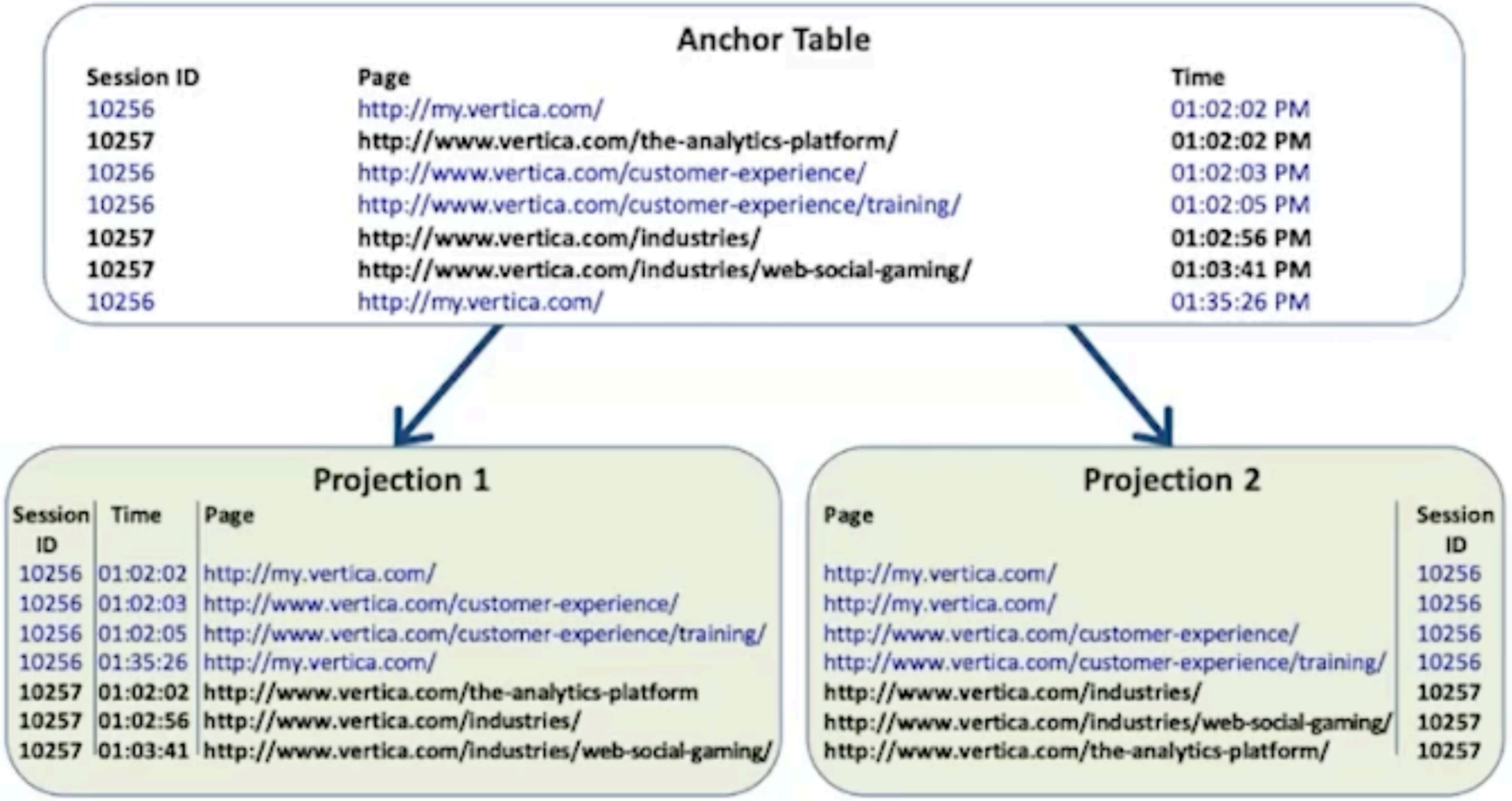


- Изначальное - пет-проект Майкла Стоунбрейкера
- Принципиальное архитектурное различие от предшественников - колоночное хранение
- Использует вместо индексов проекции - физически материализованные, отсортированные и распределённые копий данных
- Вышла на рынок в 2005, распространяется по модели лицензионных отчислений за объем хранения
- Постепенно меняла архитектуру и к 2018 пришла к формату разделенного хранения и обработки

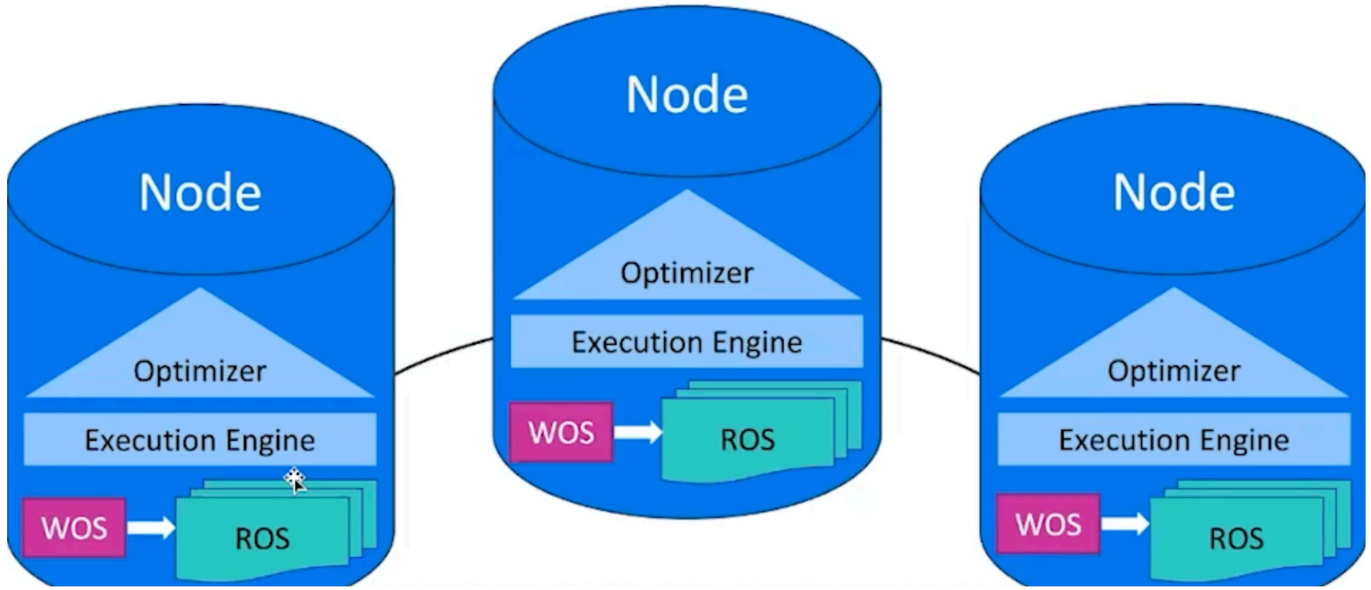
Проекции


TABLE
Logical
customer
A
B
C
PROJECTIONS
Physical
A
B
C
B
A
C
C
A
customer_p1
customer_p2
customer_p3
Node 1

Проекции


Anchor Table
Session ID
Page
Time
10256
http://my.vertica.com/
01:02:02 PM
10257
http://www.vertica.com/the-analytics-platform/
01:02:02 PM
10256
http://www.vertica.com/customer-experience/
01:02:03 PM
10256
http://www.vertica.com/customer-experience/training/
01:02:05 PM
10257
http://www.vertica.com/industries/
01:02:56 PM
10257
http://www.vertica.com/industries/web-social-gaming/
01:03:41 PM
10256
http://my.vertica.com/
01:35:26 PM
Projection 1
Session ID
Time
Page
10256 01:02:02 http://my.vertica.com/
10256 01:02:03 http://www.vertica.com/customer-experience/
10256 01:02:05 http://www.vertica.com/customer-experience/training/
10256 01:35:26 http://my.vertica.com/
10257 01:02:02 http://www.vertica.com/the-analytics-platform
10257 01:02:56 http://www.vertica.com/industries/
10257 01:03:41 http://www.vertica.com/industries/web-social-gaming/
Projection 2
Page
Session ID
http://my.vertica.com/ 10256
http://my.vertica.com/ 10256
http://www.vertica.com/customer-experience/ 10256
http://www.vertica.com/customer-experience/training/ 10256
http://www.vertica.com/industries/ 10257
http://www.vertica.com/industries/web-social-gaming/ 10257
http://www.vertica.com/the-analytics-platform/ 10257

Vertica



- По логике Enterprise версия похожа на Teradata
- Данные сначала попадают в WOS-хранилище - строчное, для быстрой записи, потом мержатся в ROS - колоночное, для быстрого чтения и эффективного хранения (до версии 9.3)
- Клиент подключается к любой ноде (initiator node), она координирует выполнение запроса; выделенного мастера нет
- Гибко администрируется, легко модифицируется и расширяется функционал

Демо 1 - Greenplum

3 - Особые случаи

Clickhouse



〉 ClickHouse = **Click**stream + Data ware**House**

〉 2009 - Алексей Миловидов в Яндексе начинает прототип

〉 2012 - работает в продакшене у Яндекс.Метрики (тогда второй по размеру сервис веб-аналитики в мире)

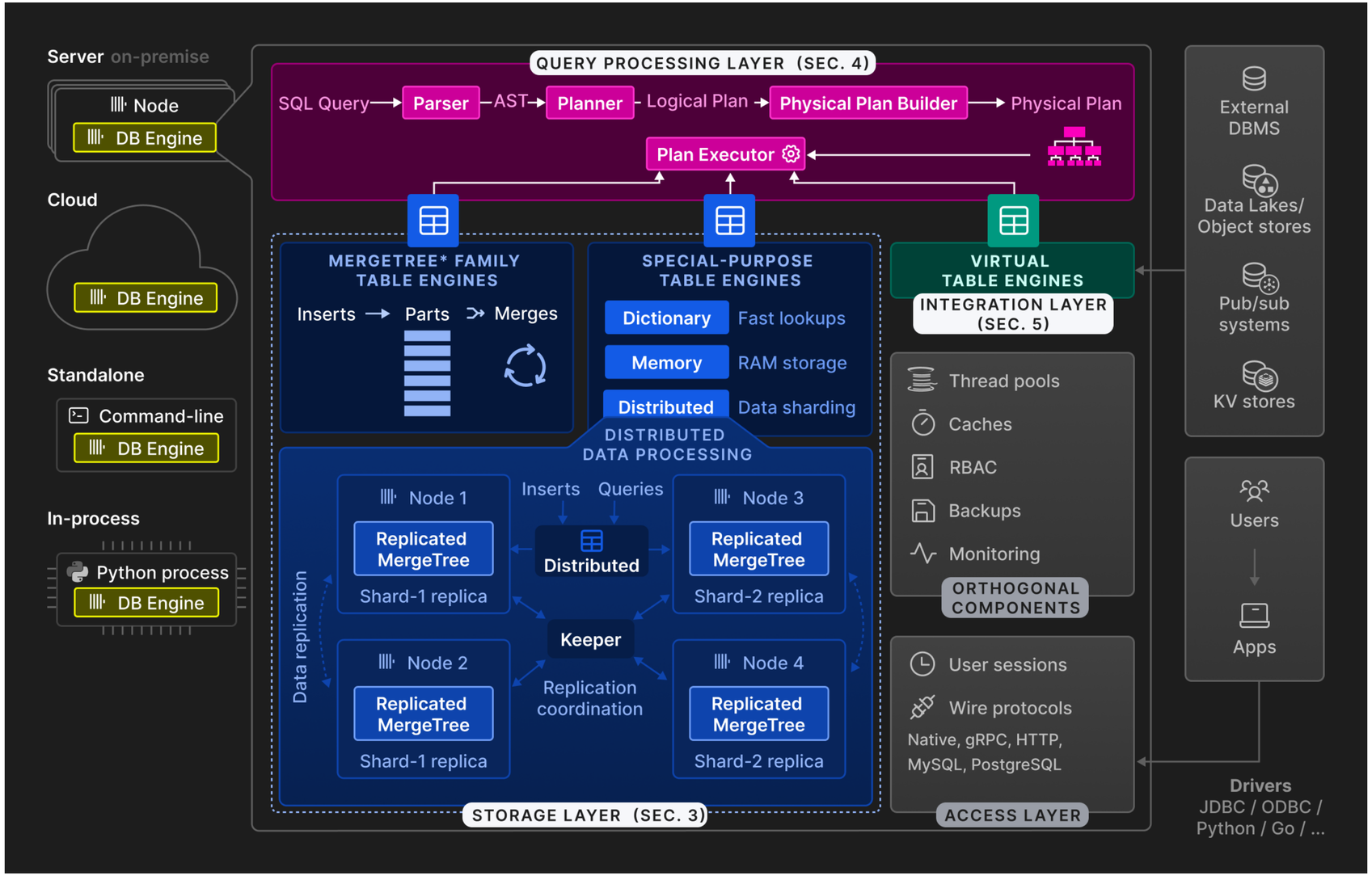
〉 Июнь 2016 - открытый код, Apache 2.0

〉 Сентябрь 2021 - выходит из Яндекса в ClickHouse, Inc.

〉 Декабрь 2022 - ClickHouse Cloud

〉 2025–2026 - раунды C ($350M) и D ($400M)

Архитектура Clickhouse



〉 Колоночная СУБД, векторное исполнение запросов

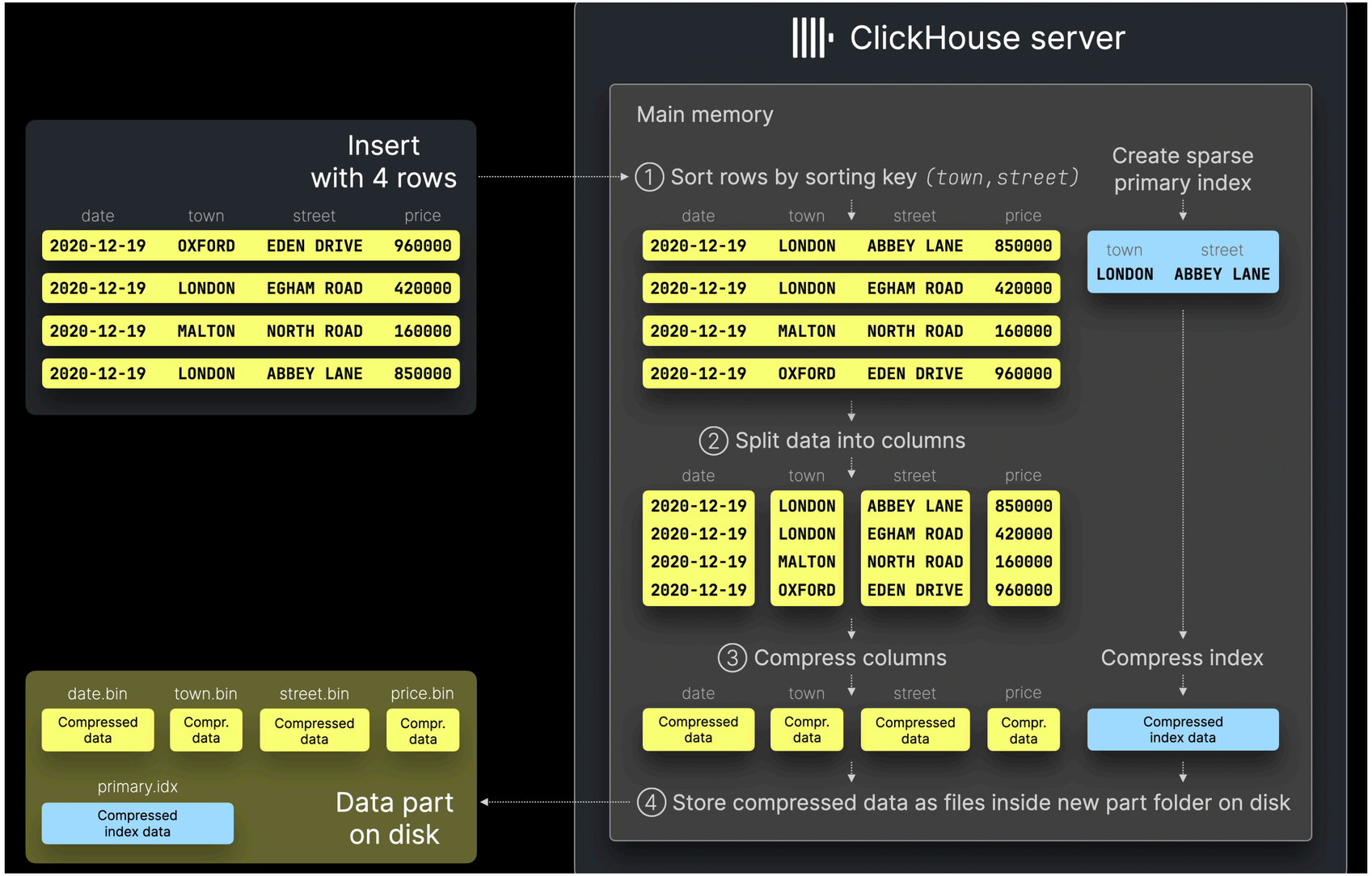
〉 Три слоя: **обработка запросов**, **хранение** (table engines), **интеграции**

〉 Табличные движки задают, как хранятся данные:

1. семейство MergeTree - основное хранение;
2. специальные: Memory, Dictionary, Distributed;
3. интеграционные: Kafka, S3, PostgreSQL, MySQL…

〉 Один и тот же движок работает на ноутбуке, на сервере, в кластере, в облаке и даже внутри процесса Python (chDB)

MergeTree



〉 Каждый INSERT создаёт новый неизменяемый part (кусок данных)

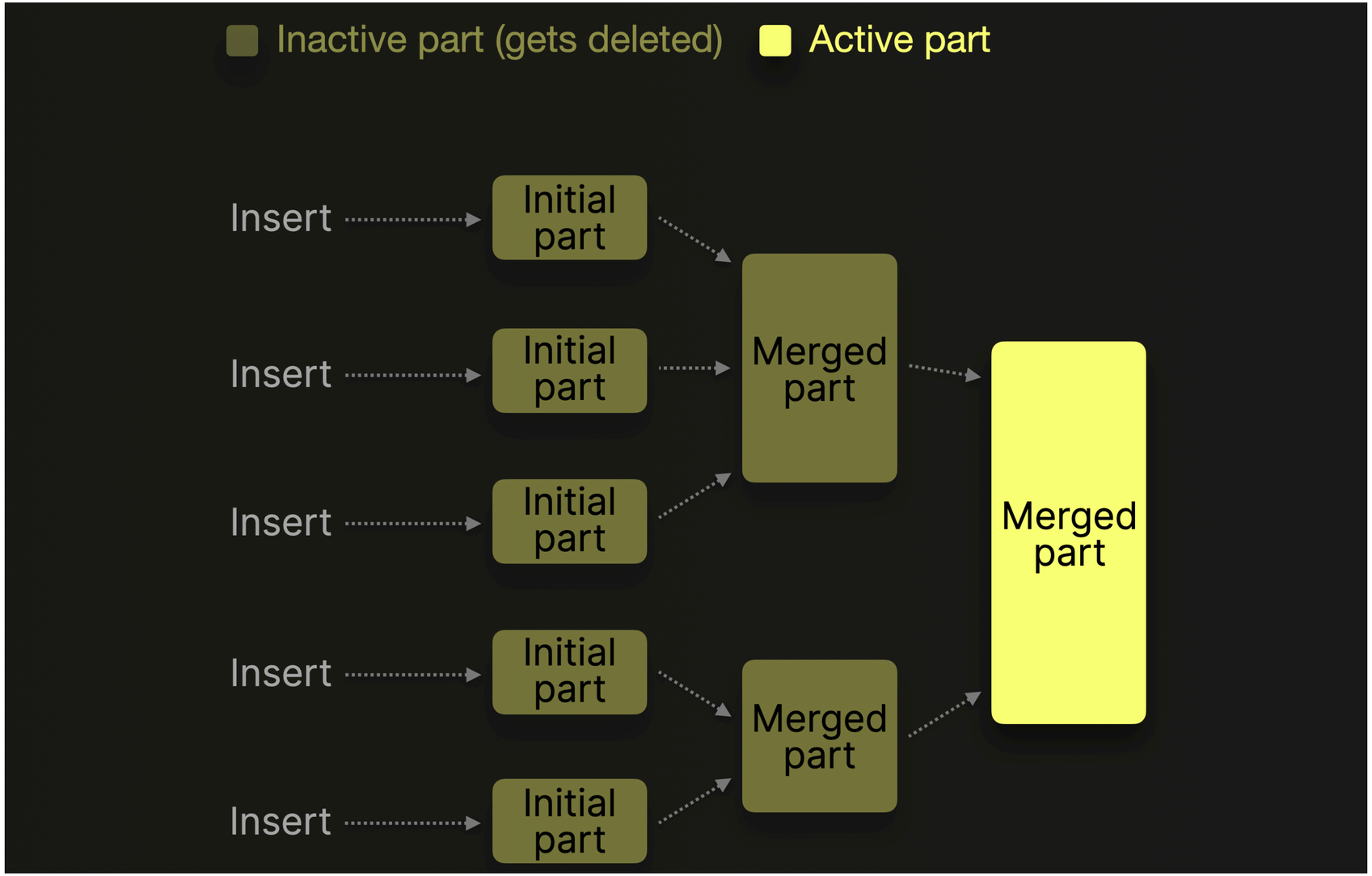
〉 Внутри парта строки отсортированы по ORDER BY, каждый столбец лежит в своём файле и сжат отдельно

〉 В фоне парты сливаются (merge) в более крупные — идея похожа на LSM-дерево

〉 Отсюда правило: вставлять пачками (тысячи и десятки тысяч строк) или через async inserts

〉 PARTITION BY нужен для управления данными (DROP / DETACH партиции), а не для распределения по нодам

MergeTree



〉 Каждый INSERT создаёт новый неизменяемый part (кусок данных)

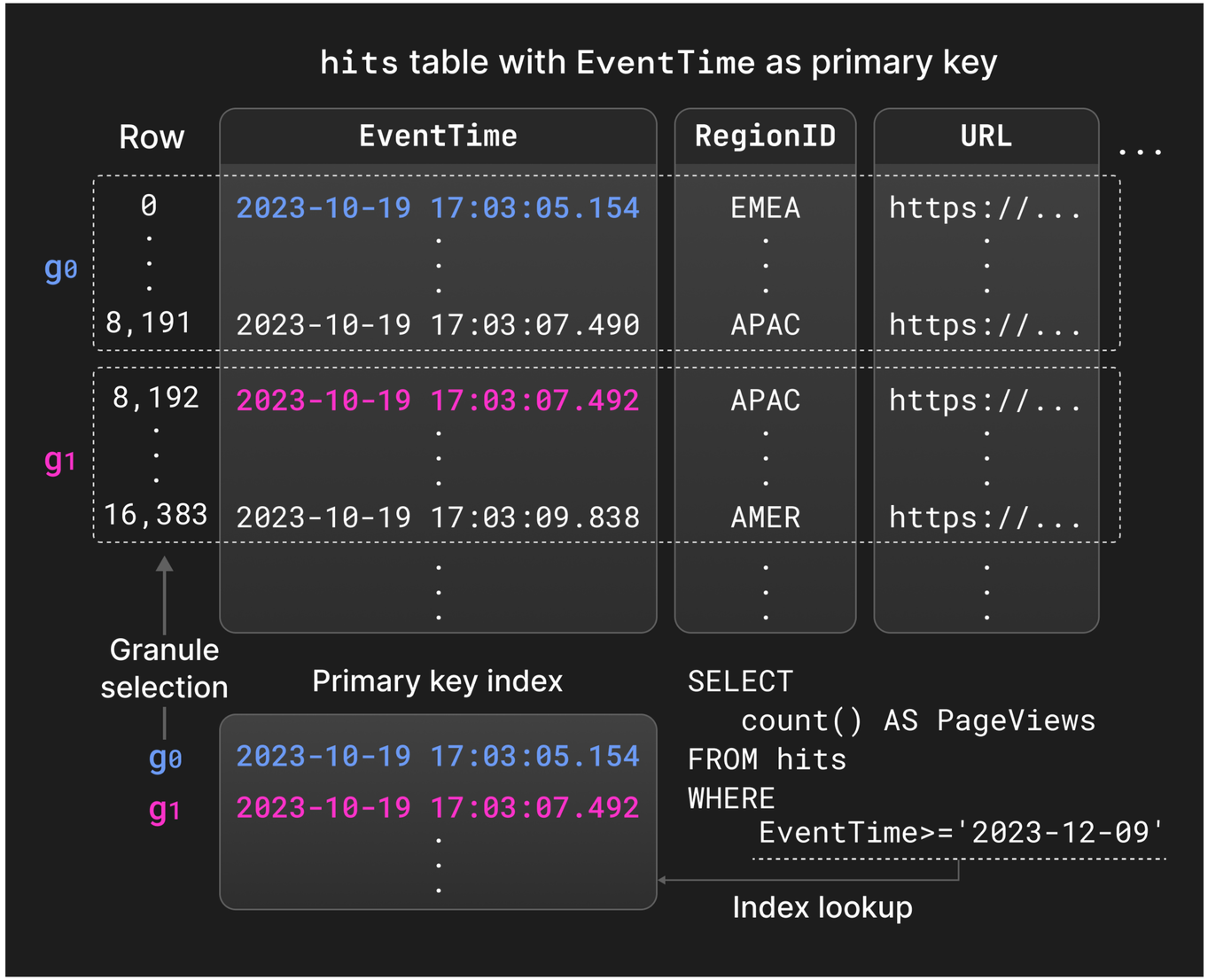
〉 Внутри парта строки отсортированы по ORDER BY, каждый столбец лежит в своём файле и сжат отдельно

〉 В фоне парты сливаются (merge) в более крупные — идея похожа на LSM-дерево

〉 Отсюда правило: вставлять пачками (тысячи и десятки тысяч строк) или через async inserts

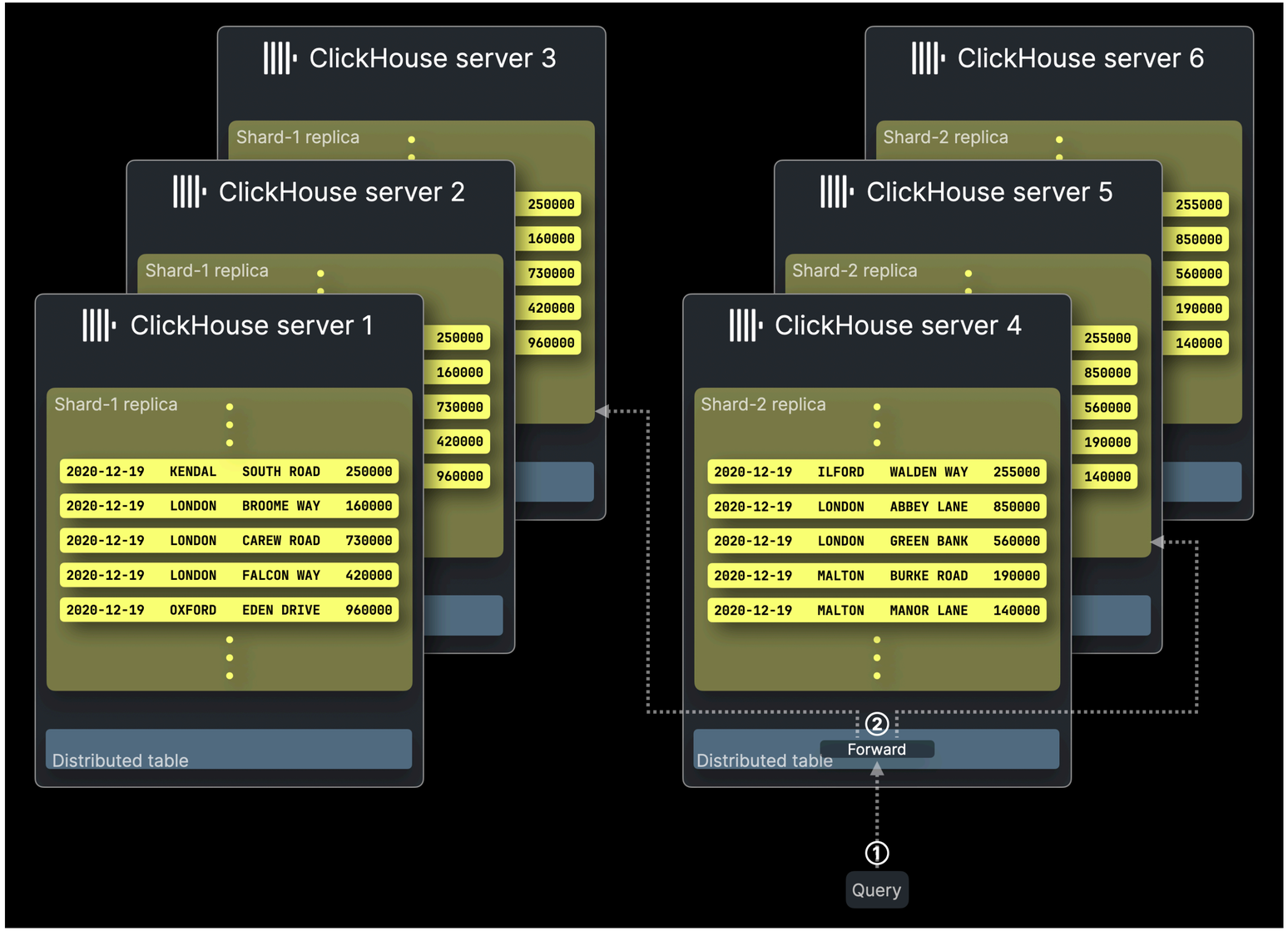
〉 PARTITION BY нужен для управления данными (DROP / DETACH партиции), а не для распределения по нодам

Разреженный первичный индекс



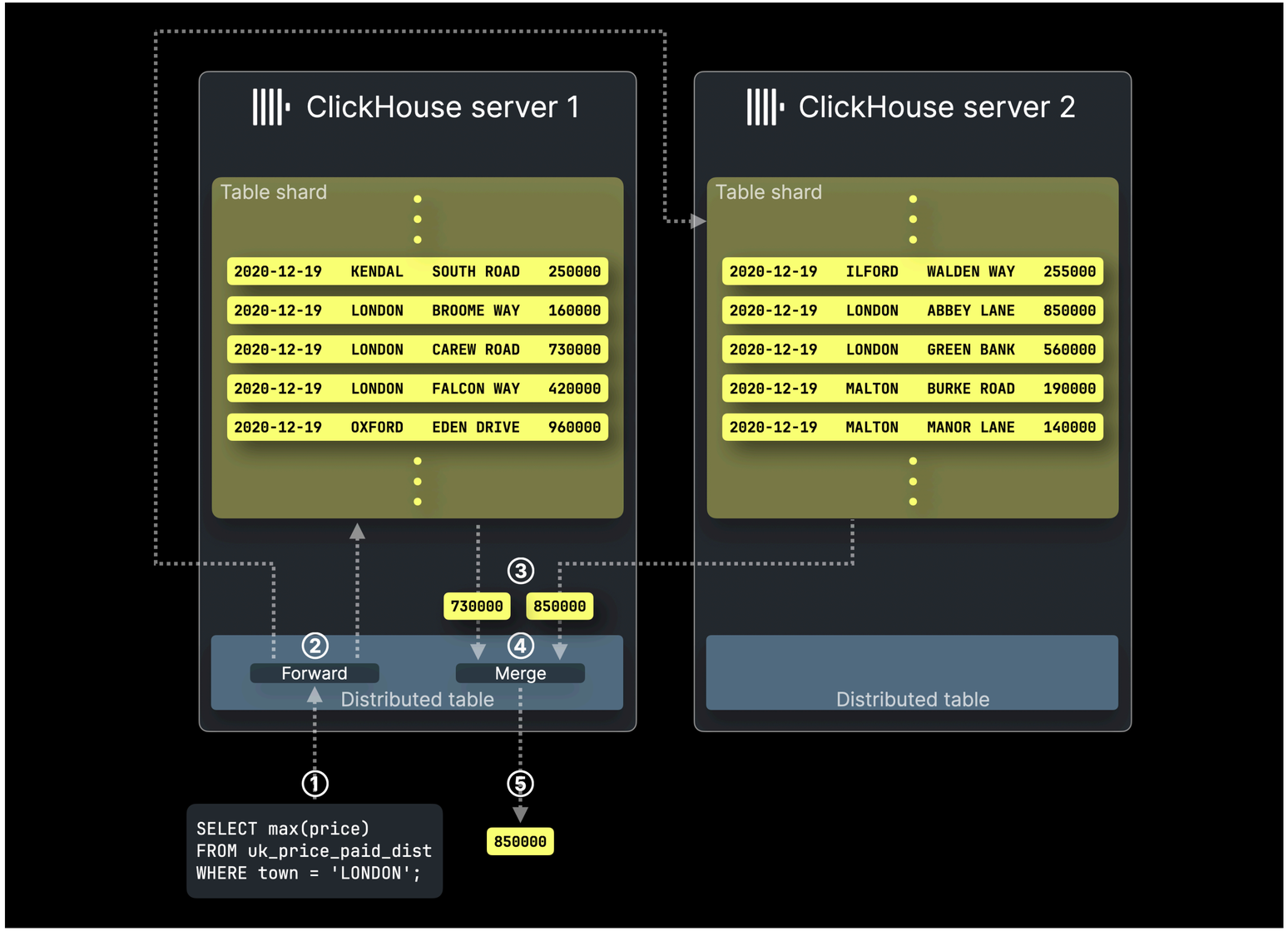
- Индекс хранит одну запись на гранулу, по умолчанию гранула — 8192 строки
- Индекс маленький и целиком лежит в памяти даже для миллиардов строк
- Запрос по ключу читает только подходящие гранулы, а не всю таблицу
- PRIMARY KEY в ClickHouse не уникален: дубликаты допустимы
- Дополнительно есть skip-индексы: minmax, set, bloom_filter

Кластер: шарды, реплики, Keeper



- Shared nothing: таблица делится на шарды, у каждого шарда есть реплики
- Репликация - движок ReplicatedMergeTree, координация через ClickHouse Keeper (или ZooKeeper)
- Multi-master: писать можно в любую реплику; репликация асинхронная
- Distributed-таблица своих данных не хранит, она роутер:
  1. INSERT раскладывает по шардам по sharding key;
  2. SELECT рассылает на все шарды и сливает результат
- Выделенного мастера нет: координатором запроса (initiator) становится нода, которая его приняла

Кластер: шарды, реплики, Keeper



- Shared nothing: таблица делится на шарды, у каждого шарда есть реплики
- Репликация - движок ReplicatedMergeTree, координация через ClickHouse Keeper (или ZooKeeper)
- Multi-master: писать можно в любую реплику; репликация асинхронная
- Distributed-таблица своих данных не хранит, она роутер:
  1. INSERT раскладывает по шардам по sharding key;
  2. SELECT рассылает на все шарды и сливает результат
- Выделенного мастера нет: координатором запроса (initiator) становится нода, которая его приняла

Плюсы и минусы

Плюсы:

〉Быстрые group by

〉Держит большие RPS

Минусы:

〉Очень не любит JOIN, особенно если права часть >10к строк

〉Из-за асинхронности мутации очень долго сходятся

〉Очень чувствителен к схеме данных и правильной настройке

Применения:

ML, BI и работа с большими и широкими таблицами без JOIN (ClickStream)

StarRocks



〉 Корни в Apache Doris:
1.проект Palo в рекламной отчётности Baidu;
2.2017 - открытый код;
3.2018 - Apache Incubator;
4.июнь 2022 - top-level project ASF

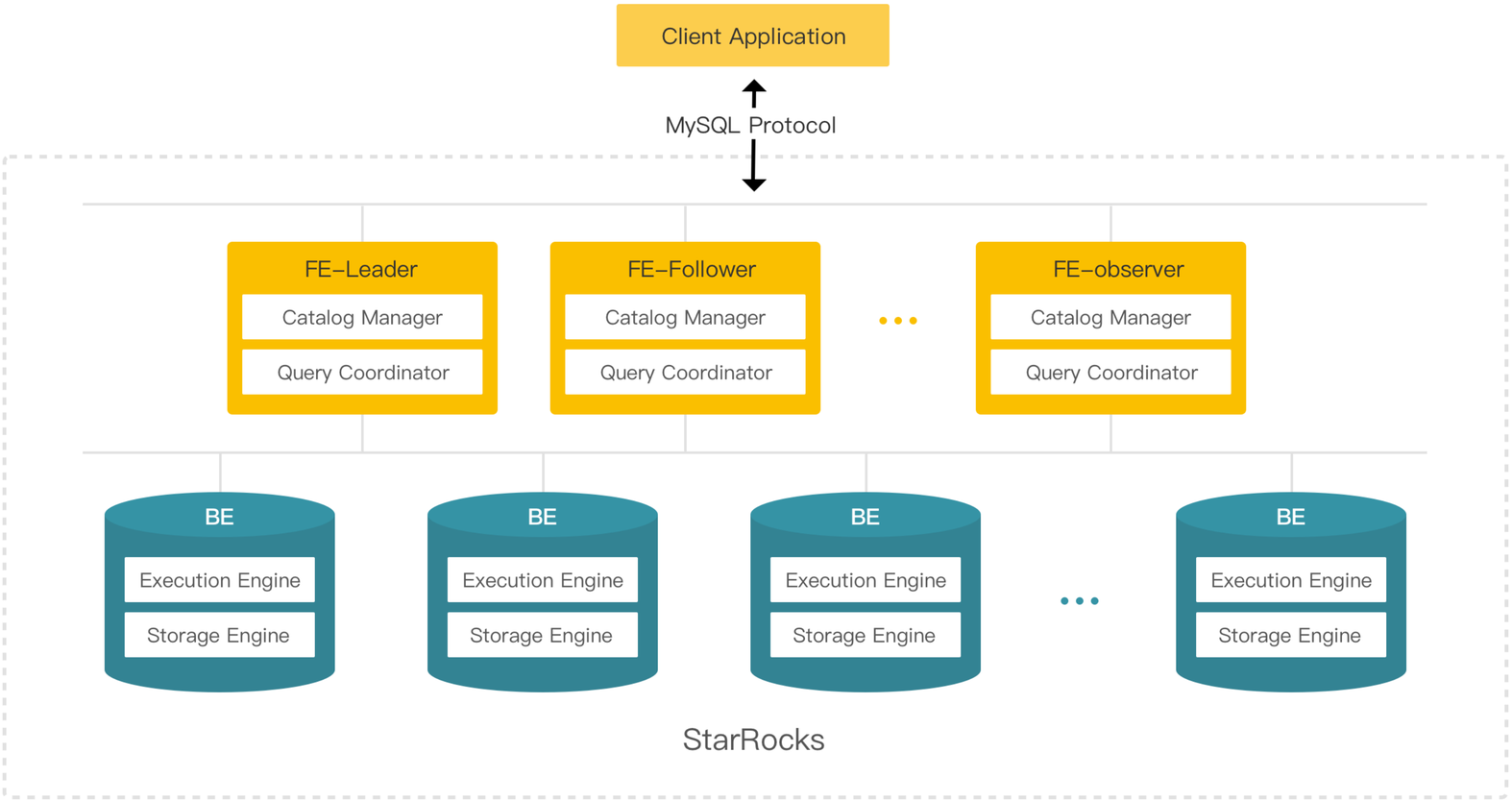
〉 2020 - часть команды Doris делает форк DorisDB

〉 2021 - после претензий к нейминг проект переименовывается в StarRocks и выходит в open source

〉 Февраль 2023 - Apache 2.0 и передача в Linux Foundation

〉 2023, версия 3.0 - архитектура shared-data

Архитектура: FE и BE



〉 Всего два типа процессов, внешних зависимостей нет (ни ZooKeeper, ни HDFS)

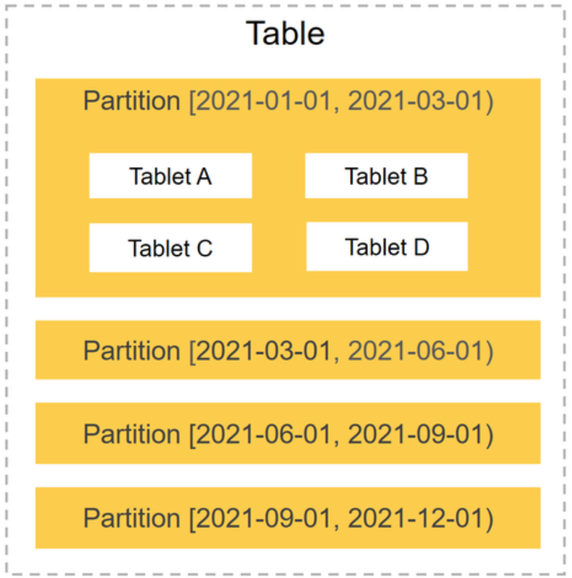
〉 FE (Frontend, Java):

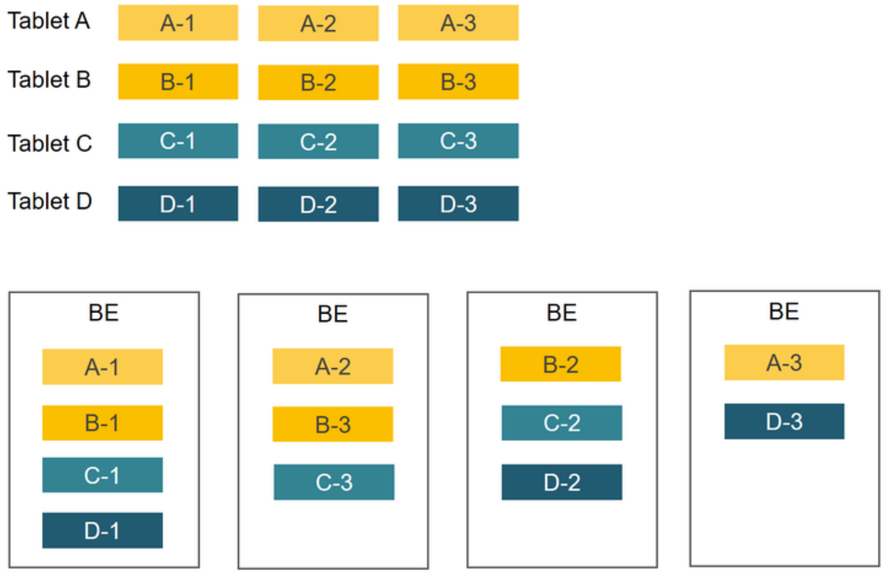
1. приём запросов по протоколу MySQL, метаданные, оптимизатор, координация;
2. роли Leader / Follower / Observer;
3. метаданные хранятся в BDB JE и реплицируются через Raft

〉 BE (Backend, C++): хранение данных и исполнение запросов

〉 Нет единой точки отказа: FE и BE масштабируются горизонтально

Распределение данных: партиции, бакеты, таблеты





〉 Двухуровневое распределение:
1.PARTITION BY - по диапазону, списку или выражению (обычно по дате);
2.DISTRIBUTED BY HASH(col) BUCKETS N (или RANDOM) - внутри партиции по бакетам

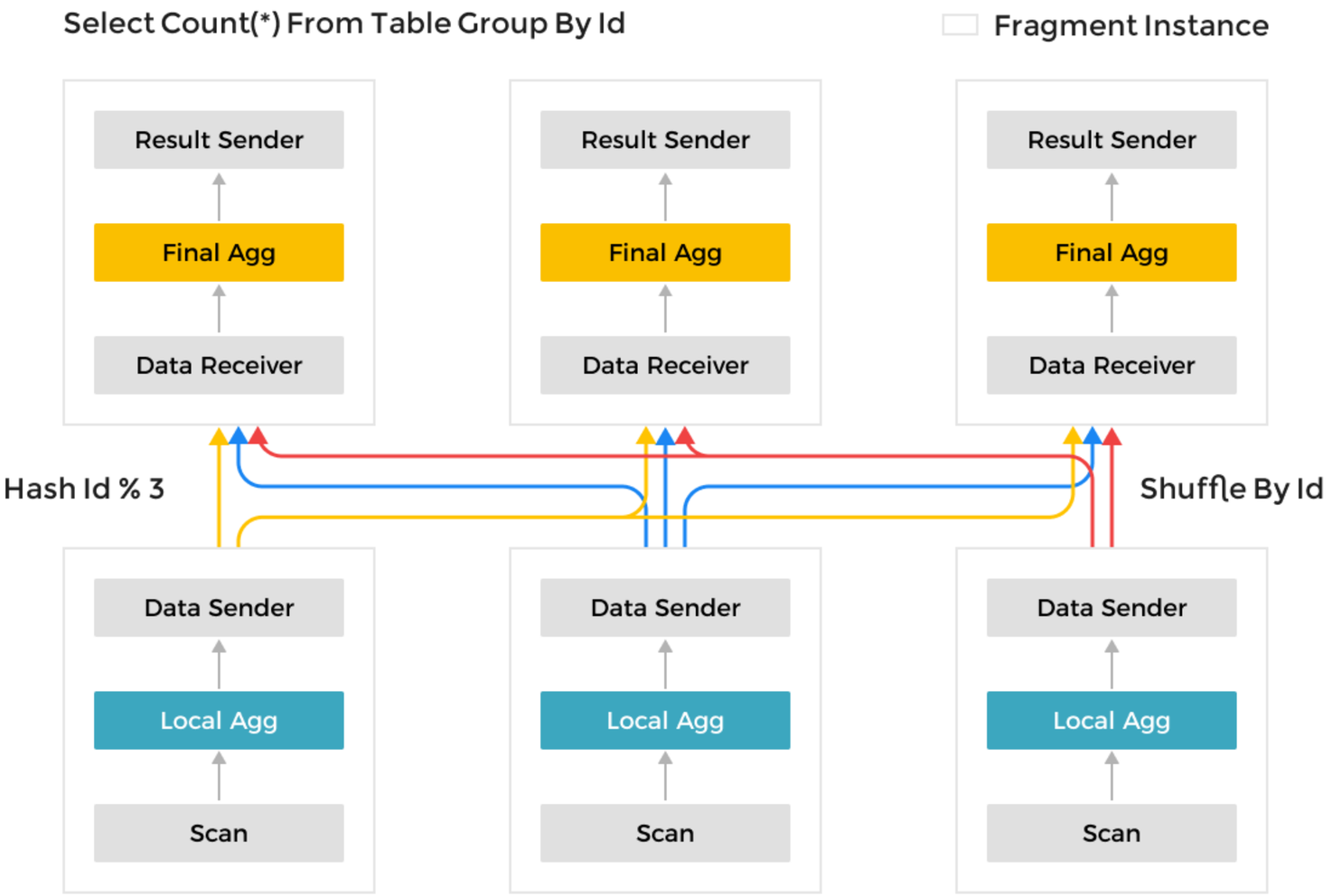
〉 Один бакет одной партиции - tablet, единица хранения, репликации и балансировки

〉 По умолчанию 3 реплики каждого таблета на разных BE

〉 Colocate group - таблицы с одинаковым ключом и числом бакетов лежат согласованно, что даёт локальный JOIN

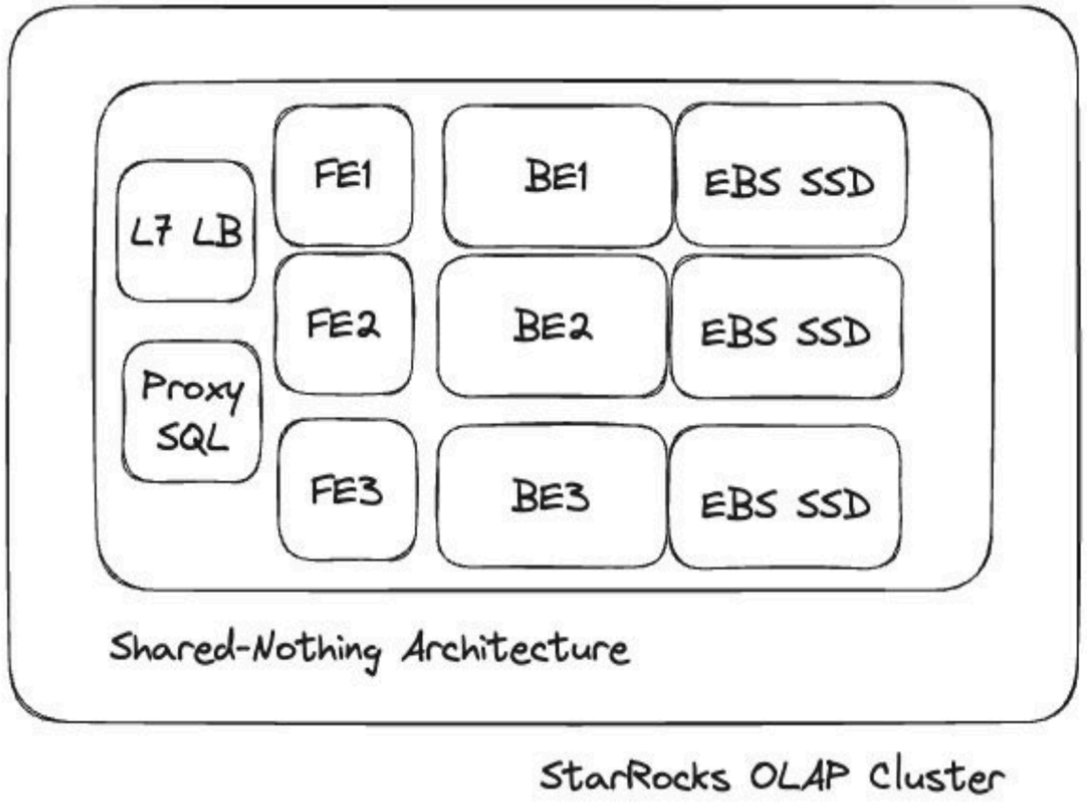
〉 Внутри таблета данные колоночные, отсортированы по ключу; есть prefix index, zone map, bloom filter

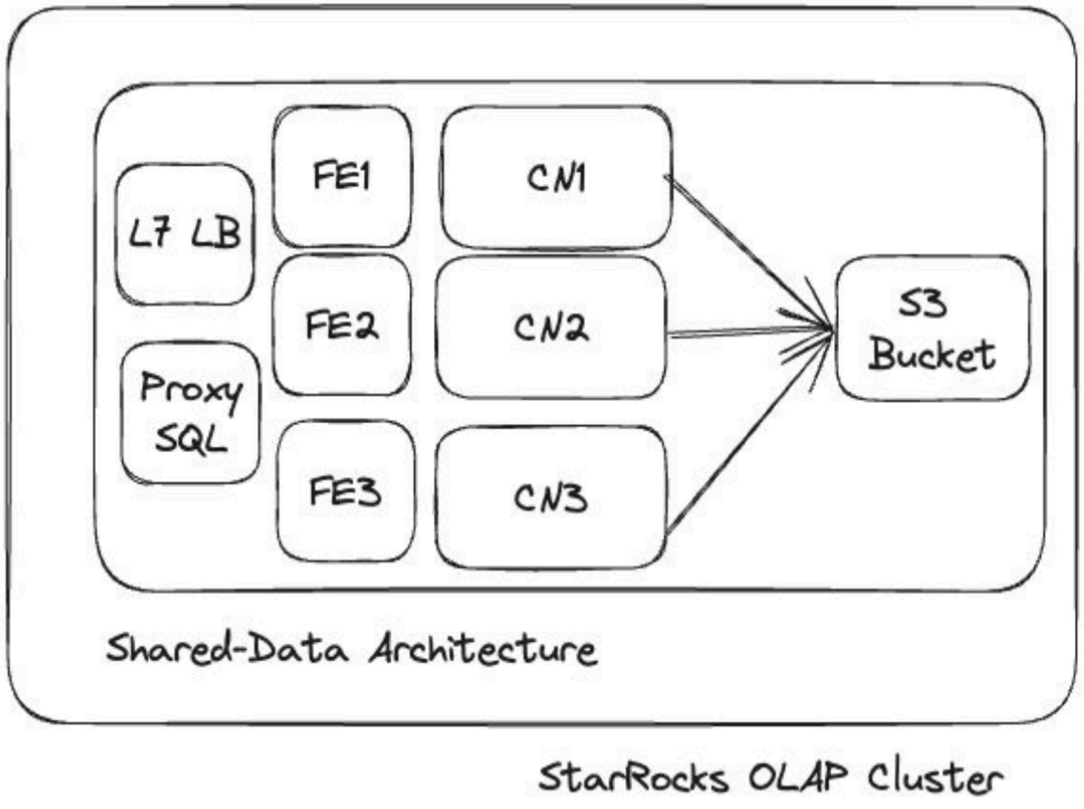
Распределение данных: партиции, бакеты, таблеты



- Запрос разбивается на фрагменты; их экземпляры выполняются параллельно на многих BE, между фрагментами данные передаются через shuffle (exchange)
- Векторный движок на C++ с SIMD
- Pipeline-исполнение для полной загрузки многоядерных CPU
- Cost-based optimizer (Cascades-подобный) на статистике — поэтому хорошо работают сложные многотабличные JOIN
- Стратегии распределённых JOIN: broadcast, shuffle, bucket shuffle, colocate

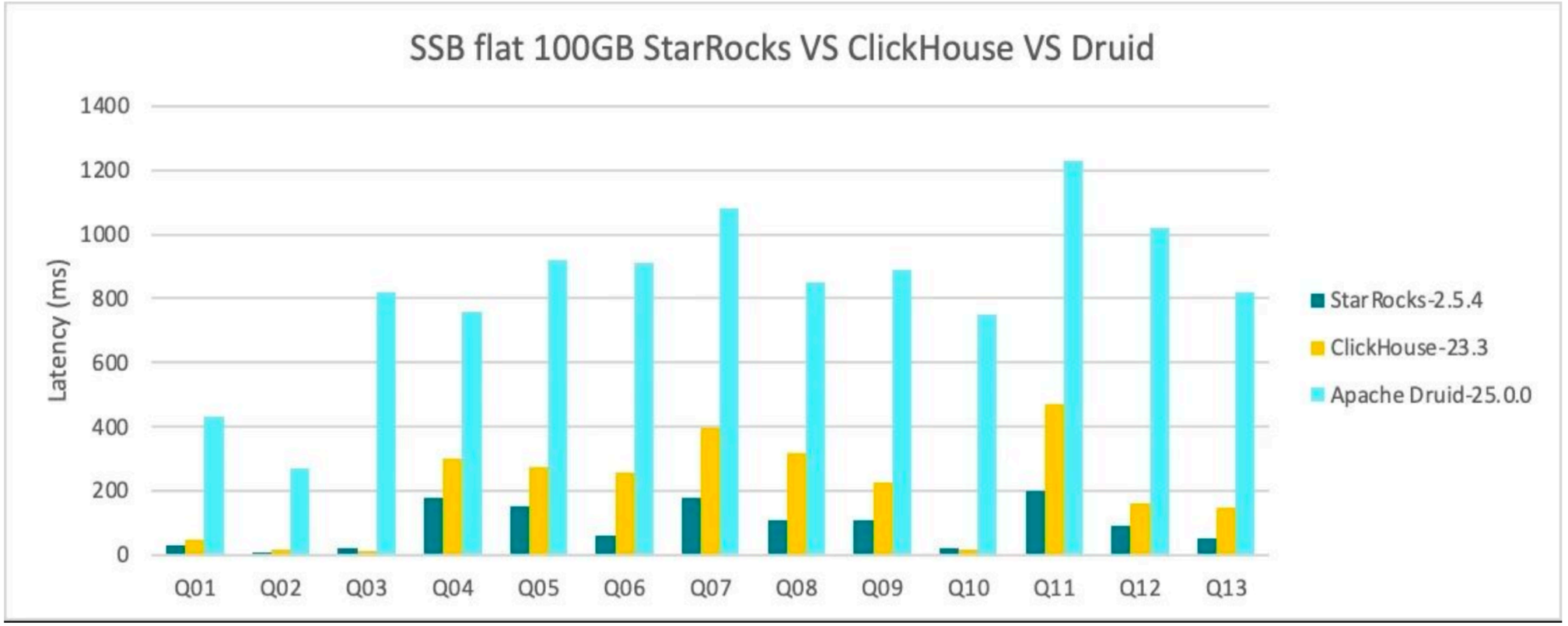
Two Deployment Architectural Choices





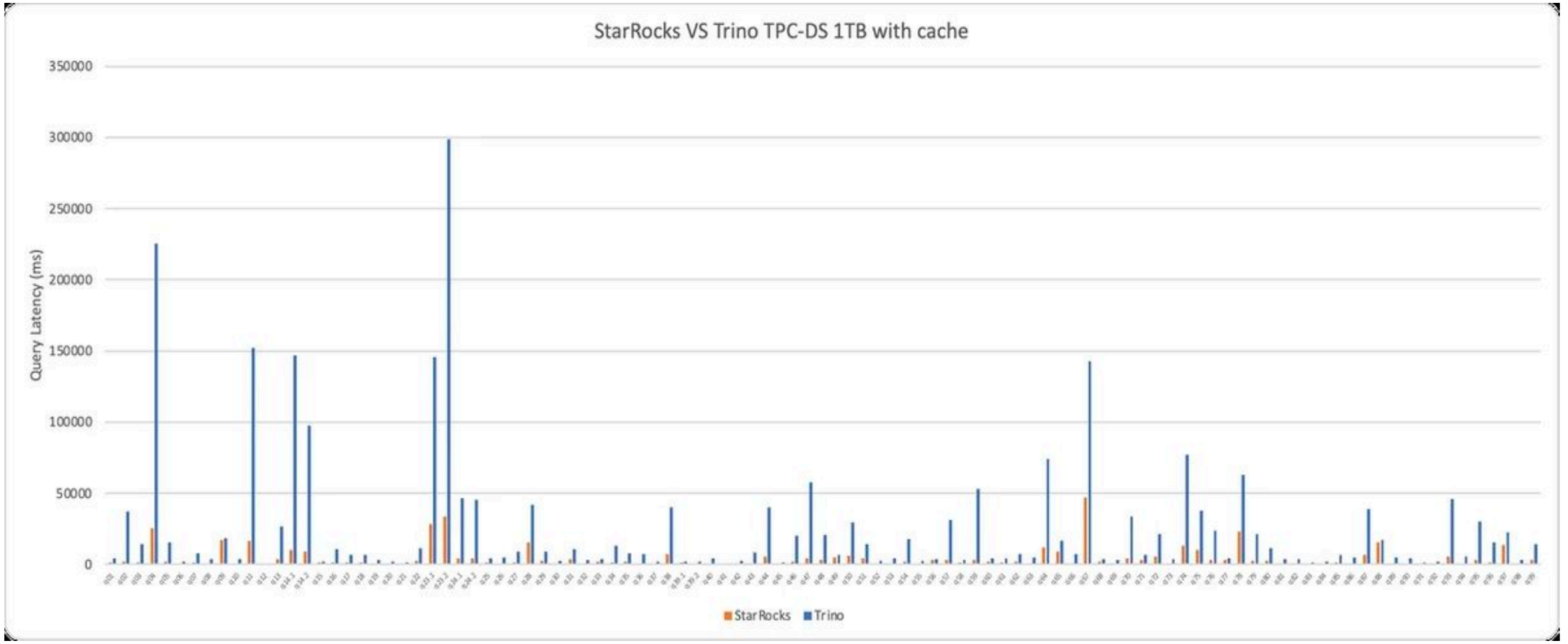
Benchmark

StarRocks Offers 2.2x Performance over ClickHouse and 8.9x Performance over Apache Druid® in Wide-table Scenarios Out of the Box using product native table format.



Benchmark

StarRocks Delivers 5.54x Query Performance over Trino in Multi-table Scenarios using Apache Iceberg table format with Parquet files.



That’s all, Folks!



Задачка

〉 У нас есть Clickhouse

〉 Особенности Clickhouse как СУБД - он очень не любит одиночные insert / update / delete, но при этом очень быстро делает group by

〉 У нас есть потребность считать агрегаты с большого объема данных - скажем, с терабайта. Clickhouse такие операции очень любит.

〉 Но данные активно меняются / дообогащаются атрибутами, что создает потребность делать много update-операций;

〉 Вопрос: можем ли мы простроить ETL так, чтобы не использовать update / delete? Если да - то как?